1. domača naloga — razlaga rešitev

Programiranje 1 · datoteka `domaca-naloga2627.ml` · vsi testi v datoteki uspešno prestani

Dokument po vrsti razloži vsako rešeno funkcijo. Rešitve sledijo slogu ostalih rešitev v repozitoriju: rekurzija z `match ... with`, pomožne notranje funkcije z apostrofom (npr. `drobiz'`), `function` za neposredno razstavljanje argumenta, posodobitev zapisov z `{ r with polje = ... }` ter funkcije iz modula `List` (`map`, `fold_left`, `filter_map`, `concat_map`).

1 Ogrevanje: vektorji

Vektorji v R³ (trojice)

```
let sestej_v_3d (x1, y1, z1) (x2, y2, z2) = (x1 +. x2, y1 +. y2, z1 +. z2)

let pomnozi_skalar_3d a (x, y, z) = (a *. x, a *. y, a *. z)

let skalarni_produkt_3d (x1, y1, z1) (x2, y2, z2) =
  (x1 *. x2) +. (y1 *. y2) +. (z1 *. z2)
```

Ker je `vec3d` trojica, jo razstavimo kar v vzorcu argumenta, npr. `(x1, y1, z1)`. Nato operacijo izvedemo po komponentah. Uporabiti moramo operatorje za števila s plavajočo vejico (`+.`, `*.`), ker OCaml ne meša `int` in `float`.

Skalarni produkt je vsota produktov istoležnih komponent: $x_1x_2 + y_1y_2 + z_1z_2$. Pri trojicah dimenzija ne more biti napačna, saj to zagotovi že tip.

Vektorji v Rⁿ (seznami)

```
let rec sestej_v u v =
  match (u, v) with
  | [], [] -> []
  | x :: u', y :: v' -> (x +. y) :: sestej_v u' v'
  | _ -> raise Neujemajoca_dimenzija

let rec pomnozi_skalar a v =
  match v with
  | [] -> []
  | x :: v' -> (a *. x) :: pomnozi_skalar a v'

let rec skalarni_produkt u v =
  match (u, v) with
  | [], [] -> 0.
  | x :: u', y :: v' -> (x *. y) +. skalarni_produkt u' v'
  | _ -> raise Neujemajoca_dimenzija
```

Seznamska različica obdela oba seznama hkrati, zato se ujemamo na paru `(u, v)`:

- `[], []` – oba vektorja sta porabljena: pri seštevanju vrnemo prazen seznam, pri skalarnem produktu nevtralni element `0.`,
- `x :: u', y :: v'` – obdelamo prvi komponenti in rekurzivno nadaljujemo na repih.
- `_` – en seznam je prazen, drugi pa ne, torej se dolžini razlikujeta in sprožimo `raise Neujemajoca_dimenzija`.
- `pomnozi_skalar` potrebuje le en seznam, zato tu napake ni mogoče dobiti.

2 Seznami: zlaganje

zlozi_levo

```
let rec zlozi_levo f acc sez =
  match sez with
  | [] -> acc
  | x :: xs -> zlozi_levo f (f acc x) xs
```

Levo zlaganje akumulator posodablja od leve proti desni: na vsakem koraku izračunamo `f acc x` in ga kot nov akumulator podamo v rekurzivni klic. Ko je seznam prazen, je akumulator rezultat. Ker je rekurzivni klic zadnja operacija, je funkcija **repno rekurzivna** in deluje enako kot `List.fold_left`.

zlozi_desno

```
let rec zlozi_desno f acc sez =
  match sez with
  | [] -> acc
  | x :: xs -> f x (zlozi_desno f acc xs)
```

Desno zlaganje najprej zloži rep seznama in šele nato uporabi `f` na glavi in tem rezultatu, kar da `f x1 (f x2 (... (f xn b0)))`. Funkcija ni repno rekurzivna (po klicu mora še izračunati `f x ...`), ustreza pa `List.fold_right`.

zlozi_levo_pretvori

```
let rec zlozi_levo_pretvori f acc sez =
  match sez with
  | [] -> (acc, [])
  | x :: xs ->
      let acc', y = f acc x in
      let acc'', ys = zlozi_levo_pretvori f acc' xs in
      (acc'', y :: ys)
```

Funkcija hkrati zlaga in preslikava. Na glavi `x` pokličemo `f acc x` in dobimo nov akumulator `acc'` ter pretvorjen element `y`. Z `acc'` nadaljujemo na repu, ki vrne končni akumulator `acc''` in seznam `ys`. Rezultat je `(acc'', y :: ys)`.

Akumulator teče od leve proti desni (tako kot pri `zlozi_levo`), pretvorjeni elementi pa ostanejo v prvotnem vrstnem redu, ker `y` dodamo na začetek že pretvorjenega repa. Funkcija je enakovredna `List.fold_left_map`.

zlozi_levo2

```
let rec zlozi_levo2 f acc sez1 sez2 =
  match (sez1, sez2) with
  | [], [] -> acc
  | x :: xs, y :: ys -> zlozi_levo2 f (f acc x y) xs ys
  | _ -> raise Neujemajoca_dimenzija
```

Gre za levo zlaganje po dveh seznamih hkrati. Ujemanje na paru je enako kot pri `sestej_v`: dva prazna seznama vrneta akumulator, dve neprazni glavi ga posodobita s `f acc x y`, vse drugo pomeni različni dolžini in sproži `Neujemajoca_dimenzija`.

3 Uporaba zlaganja

norma

```
let norma v = Float.sqrt (zlozi_levo (fun acc x -> acc +. (x *. x)) 0. v)
```

Brez `let rec`: z `zlozi_levo` seštejemo kvadrate komponent, začnemo z `0.`, na koncu pa uporabimo `Float.sqrt` (tako kot v rešitvah 1. vaj). Za prazen vektor dobimo $\sqrt{0} = 0$.

skalarni_produkt_zlozi

```
let skalarni_produkt_zlozi u v =
  zlozi_levo2 (fun acc x y -> acc +. (x *. y)) 0. u v
```

Z `zlozi_levo2` sproti prištevamo `x *. y`. Preverjanje dimenzij dobimo zastonj, saj izjemo sproži že `zlozi_levo2`.

4 Prodajni avtomat

Vsi zneski so cela števila v centih, zato ni zaokrožitvenih napak.

vrednost

```
let vrednost = function
  | C10 -> 10
  | C20 -> 20
  | C50 -> 50
  | E1 -> 100
  | E2 -> 200
```

Z `function` neposredno ločimo primere po konstruktorju kovanca. Ker so našteti vsi konstruktorji, je ujemanje izčrpno in prevajalnik bi nas opozoril, če bi kdaj dodali nov kovanec.

vsota

```
let vsota kovanci = List.fold_left (fun acc k -> acc + vrednost k) 0 kovanci
```

Seznam kovancev zložimo z `List.fold_left`: akumulatorju prištevamo vrednost vsakega kovanca. Prazen seznam ima vsoto 0.

drobiz

```
let drobiz znesek =
  let kovanci = [ E2; E1; C50; C20; C10 ] in
  let rec drobiz' znesek kovanci =
    match kovanci with
    | [] -> []
    | k :: ks ->
        if vrednost k <= znesek then
          k :: drobiz' (znesek - vrednost k) kovanci
        else drobiz' znesek ks
  in
  drobiz' znesek kovanci
```

Požrešni algoritem. Kovance hranimo v seznamu, urejenem od največjega proti najmanjšemu. Pomožna funkcija `drobiz'` gleda prvi kovanec `k`:

- če je `vrednost k <= znesek`, kovanec vzamemo in nadaljujemo z zmanjšanim zneskom, **pri čemer seznam kovancev ostane cel**, saj lahko isti kovanec uporabimo večkrat;
- sicer je kovanec prevelik, zato ga zavržemo in poskusimo z manjšimi (`ks`);
- ko kovancev zmanjka, je znesek nujno 0 (ker je deljiv z 10 in imamo `C10`), zato vrnemo `[]`.

Ker za vsak tip kovanca porabimo največje možno število, dobimo najmanjše število kovancev, izhod pa je urejen padajoče. Primer: 250 → `[E2; C50]`.

v_niz

```
let v_niz znesek = Printf.sprintf "%d.%02d €" (znesek / 100) (znesek mod 100)
```

Celoštevilsko deljenje s 100 da evre, ostanek `mod 100` pa cente. Format `%02d` poskrbi, da imajo centi vedno dve števki (5 → "05"), npr. `v_niz 1205 = "12.05 €"`.

poisci

```
let rec poisci ime zaloga =
  match zaloga with
  | [] -> None
  | izd :: zaloga' -> if izd.ime = ime then Some izd else poisci ime zaloga'
```

Rekurzivno sprehajanje po zalogi: če ima trenutni izdelek iskano ime, vrnemo `Some izd`, sicer iščemo naprej. Na koncu praznega seznama izdelka ni, zato vrnemo `None`. Tip `option` eksplicitno pove, da iskanje lahko ne uspe.

spremeni_kolicino

```
let spremeni_kolicino ime n zaloga =
  List.map
    (fun izd ->
      if izd.ime = ime then { izd with kolicina = izd.kolicina + n } else izd)
    zaloga
```

Z `List.map` gremo čez vse izdelke. Izdelek z danim imenom nadomestimo s kopijo, ki ima posodobljeno količino (`{ izd with kolicina = ... }`), ostale vrnemo nespremenjene. Če imena ni v zalogi, se tako nič ne spremeni.

opis_napake

```
let opis_napake = function
  | Premalo n -> "Vstavite še " ^ v_niz n
  | NiIzdelka ime -> "Izdelka " ^ ime ^ " ni"
  | Razprodano ime -> "Izdelek " ^ ime ^ " je razprodan"
  | VVzdrzevanju -> "Avtomat je v vzdrževanju"
  | NiVVzdrzevanju -> "Avtomat ni v vzdrževanju"
```

Vsak konstruktor napake pretvorimo v niz z lepljenjem nizov (^). Pri `Premalo` znesek oblikujemo s prej napisano funkcijo `v_niz`.

korak – osrednja funkcija avtomata

```
let korak stanje vnos =
  match (stanje.nacin, vnos) with
  | Obratuje, Vstavi k ->
      let kredit = stanje.kredit + vrednost k in
      ({ stanje with kredit }, Prikazi kredit)
  | Obratuje, Izberi ime -> (
      match poisci ime stanje.zaloga with
      | None -> (stanje, Napaka (NiIzdelka ime))
      | Some izd when izd.kolicina = 0 -> (stanje, Napaka (Razprodano ime))
      | Some izd when stanje.kredit < izd.cena ->
          (stanje, Napaka (Premalo (izd.cena - stanje.kredit)))
      | Some izd ->
          ( {
              stanje with
              kredit = 0;
              zaloga = spremeni_kolicino ime (-1) stanje.zaloga;
            },
            Izdaj
              {
                izdelek = ime;
                cena = izd.cena;
                drobiz = drobiz (stanje.kredit - izd.cena);
              } ))
  | Obratuje, Preklici ->
      ({ stanje with kredit = 0 }, Vrni (drobiz stanje.kredit))
  | Obratuje, ZacniVzdrzevanje ->
      ( { stanje with nacin = Vzdrzevanje; kredit = 0 },
        Vrni (drobiz stanje.kredit) )
  | Obratuje, (KoncajVzdrzevanje | Dopolni _) -> (stanje, Napaka NiVVzdrzevanju)
  | Vzdrzevanje, Vstavi k -> (stanje, Vrni [ k ])
  | Vzdrzevanje, (Izberi _ | Preklici | ZacniVzdrzevanje) ->
      (stanje, Napaka VVzdrzevanju)
  | Vzdrzevanje, KoncajVzdrzevanje ->
      ({ stanje with nacin = Obratuje }, Prikazi stanje.kredit)
  | Vzdrzevanje, Dopolni (ime, n) -> (
      match poisci ime stanje.zaloga with
      | None -> (stanje, Napaka (NiIzdelka ime))
      | Some _ ->
          ( { stanje with zaloga = spremeni_kolicino ime n stanje.zaloga },
            Prikazi stanje.kredit ))
```

Obnašanje je odvisno od načina in od vnosa, zato se ujemamo na paru `(stanje.nacin, vnos)`. Tako je vsaka vrstica tabele iz navodil en primer v `match`, prevajalnik pa preveri, da smo pokrili vse kombinacije. Funkcija vedno vrne par (novo stanje, izhod); stanja ne spreminjamo, temveč z `{ stanje with ... }` ustvarimo novega.

Ko avtomat obratuje:

- `Vstavi k` – novi kredit izračunamo enkrat in ga uporabimo za stanje (`{ stanje with kredit }` je okrajšava za `kredit = kredit`) in za izhod `Prikazi`.
- `Izberi ime` – izdelek poiščemo s `poisci` in v gnezdenem `match` s stražami `when` preverimo pogoje v vrstnem redu iz navodil: ni izdelka → `NiIzdelka`; količina 0 → `Razprodano`; premalo kredita → `Premalo` z razliko. Pri napaki vrnemo nespremenjeno stanje. Sicer kredit postavimo na 0, zalogo zmanjšamo s `spremeni_kolicino ime (-1)`, drobiž pa izračunamo z `drobiz (kredit - cena)`. Gnezdeni `match` je v oklepajih, da ne bi pogoltnil naslednjih primerov zunanjega.
- `Preklici` – ves kredit vrnemo kot kovance (`drobiz`) in ga postavimo na 0.
- `ZacniVzdrzevanje` – podobno kot preklic, le da hkrati preklopimo način na `Vzdrzevanje`. Pri kreditu 0 je izhod `Vrni []`.
- `KoncajVzdrzevanje | Dopolni _` – z or-vzorcem oba primera obdelamo skupaj: napaka `NiVVzdrzevanju`.

Ko je avtomat v vzdrževanju:

- `Vstavi k` – kovanec takoj vrnemo (`Vrni [ k ]`), stanje se ne spremeni.
- `Izberi _ | Preklici | ZacniVzdrzevanje` – napaka `VVzdrzevanju`.
- `KoncajVzdrzevanje` – način nazaj na `Obratuje`, izhod prikaže kredit.
- `Dopolni (ime, n)` – najprej s `poisci` preverimo, ali izdelek obstaja (`spremeni_kolicino` bi neznano ime tiho prezrla); če obstaja, povečamo količino za `n` in prikažemo kredit, sicer javimo `NiIzdelka`.

simuliraj

```
let simuliraj stanje vnosi = zlozi_levo_pretvori korak stanje vnosi
```

Simulacija je natanko levo zlaganje s pretvorbo: akumulator je stanje avtomata, vsak vnos pa se pretvori v izhod. Tip `korak` (`stanje -> vnos -> stanje * izhod`) se ujema s tipom funkcije, ki jo pričakuje `zlozi_levo_pretvori`, zato ga lahko podamo neposredno. Rezultat je par (končno stanje, seznam vseh izhodov v vrstnem redu vnosov).

5 Analiza izhodov

promet

```
let promet izhodi =
  List.fold_left
    (fun acc izhod ->
      match izhod with Izdaj { cena; _ } -> acc + cena | _ -> acc)
    0 izhodi
```

Z `List.fold_left` seštejemo cene. Izhod `Izdaj` ima inline zapis, zato iz njega z vzorcem `Izdaj { cena; _ }` vzamemo le polje `cena` (`_` izpusti ostala polja). Vsi drugi izhodi akumulatorja ne spremenijo.

napake

```
let napake izhodi =
  List.filter_map (function Napaka n -> Some n | _ -> None) izhodi
```

`List.filter_map` hkrati filtrira in preslika: za `Napaka n` vrne `Some n` (element obdržimo), za vse drugo `None` (element zavržemo). Vrstni red se ohrani.

izplacani_kovanci

```
let izplacani_kovanci izhodi =
  List.concat_map
    (function
      | Izdaj { drobiz; _ } -> drobiz
      | Vrni kovanci -> kovanci
      | _ -> [])
    izhodi
```

`List.concat_map` vsak izhod preslika v seznam kovancev in rezultate stakne: pri `Izdaj` je to drobiž, pri `Vrni` vrnjeni kovanci, pri ostalih izhodih prazen seznam. Tako dobimo vse izplačane kovance v vrstnem redu izplačila.

prodano

```
let prodano izhodi =
  let rec dodaj ime = function
    | [] -> [ (ime, 1) ]
    | (ime', n) :: ostali when ime' = ime -> (ime', n + 1) :: ostali
    | par :: ostali -> par :: dodaj ime ostali
  in
  List.fold_left
    (fun acc izhod ->
      match izhod with Izdaj { izdelek; _ } -> dodaj izdelek acc | _ -> acc)
    [] izhodi
```

Pomožna funkcija `dodaj` v seznam parov `(ime, število)` doda eno prodajo: če par z imenom že obstaja, mu poveča števec, sicer na konec doda `(ime, 1)`. Ker nove izdelke dodajamo na konec, so izdelki urejeni po vrstnem redu prve prodaje. Z `List.fold_left` gremo čez izhode in za vsak `Izdaj { izdelek; _ }` pokličemo `dodaj`.

6 Preverjanje

Datoteko poženemo z `ocaml domaca-naloga2627.ml`. Vseh 27 testnih sklopov (skupaj 137 primerov) je uspešnih (`Incorrect: 0`). Dodatno so preverjeni primeri iz navodil, npr. `simuliraj zacetno_stanje primer_vnosi` vrne pričakovano končno stanje (4 čokolade) in izhode, `promet` teh izhodov je 240, `izplacani_kovanci` pa `[C50; C20; C10; C50]`.